Design a smart-city emergency routing system with 50 +intersections (Nodes) and 100 weighted roads (Edges and Weights)

Submitted in the partial fulfilment for the Course

of

CSA0311- Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

BTECH IN COMPUTER SCIENCE AND ENGINEERING

Submitted by

Lingeshwar T(192521226)

Finny franklin R(192511450)

P.Sumanth Kumar(192472054)

Under the Supervision of

Dr.K.Kiruthika

Saveetha School of engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

DESIGN A SMART-CITY EMERGENCY ROUTING SYSTEM

with 100+ Intersections (Nodes) and 200+ Weighted Roads (Edges and Weights)

Submitted in the partial fulfilment for the Course

CSA0311 – Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

BTECH IN COMPUTER SCIENCE AND ENGINEERING

Submitted by

Lingeshwar T(192521226)

Finny franklin R(192511450)

P.Sumanth Kumar(192472054)

Under the Supervision of

Dr. K. Kiruthika

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

Design a Smart-City Emergency Routing System with 100+ Intersections and 200+ Weighted Roads

Assignment Information

Category	Detail
Department	Computer Science and Engineering/Information Technology
Programme	B.Tech in Computer Science and Engineering
Course Code & Course Name	CSA0311 — Data Structures
Academic Year / Batch	2025–2029
Faculty Name	Dr. K. Kiruthika
Assignment Title	Design a Smart-City Emergency dispatch System with 100+ Intersections and 200+ Weighted Roads
Date of Issue	30.8.2026
Date of Submission	02 September 2026
Maximum Marks	100
Course Outcome(s) – CO	Apply appropriate data structures and algorithms to model and solve a real-world graph-based problem
Bloom's Taxonomy Level	L4/L5 — Analyze, Evaluate (design and justify a solution against constraints)
SDG Mapping (SDG 1-17)	SDG 3 — Good Health and Well-being; SDG 11 — Sustainable Cities and Communities
Industry / Societal Relevance	Real-time dispatch routing is directly used by ambulance, fire, and police services, and underlies commercial mapping/navigation platforms

1. ASSIGNMENT PROBLEM / CHALLENGE

A realistic computing design challenge: model a city-scale road network as a weighted graph and build a routing engine that finds the shortest travel-time route for an emergency vehicle (ambulance, fire engine) between any two intersections, under the constraints of scale, latency, and frequently changing road conditions. The assignment requires applying data-structure and algorithm concepts — graph representation, priority queues, hashing — to a problem with real performance requirements, comparing at least one alternative algorithmic approach, using a compiler/build toolchain and a browser as modern tools, and interpreting measured timing results against a stated budget.

2. PROBLEM STATEMENT

PROBLEM / CASE / DESIGN CHALLENGE

Design a smart-city emergency routing system that models the road network as a graph with more than 500 intersections (nodes) and more than 200 weighted roads (edges), where each edge weight represents travel time. The system must:

- Return the shortest available route (by travel time, not hop count) between any station and any incident location.
- Compute each route within a strict 200-millisecond budget.
- Absorb frequent live updates to road travel times (congestion, closures) without an expensive rebuild.
- Keep memory usage moderate — no data structure that scales quadratically with network size.
- Support both a single one-to-one query and repeated queries from different source stations to the same or different destinations.

3. OBJECTIVE

The objective of this project is to design, implement, and evaluate a real-time shortest-path routing engine for emergency vehicles that:

- Represents a city road network efficiently using data structures built from first principles (no external graph or algorithm libraries).
- Computes the shortest travel-time route between any station and any incident location using a correct, well-known weighted shortest-path algorithm.
- Supports live updates to road travel times in O(1) average time, so the routing layer always reflects current traffic conditions.
- Meets a hard 200 ms query budget, verified empirically at a scale beyond the assignment's minimum intersection/road requirement.
- Is demonstrated through two complementary deliverables: a console-based C engine that prints timing and route data, and an interactive, browser-based EMS dispatch console that visualises the same algorithm on a small city grid.

4. REQUIREMENTS AND CONSTRAINTS

Category	Requirement / Constraint
Functional requirement	Given a source intersection and a destination intersection, return the minimum-travel-time path and its total travel time
Performance requirement	Route query must complete in under 200 ms; live traffic update must not require rebuilding the whole graph
Technical constraint	Graph must be sparse-friendly (adjacency list, not matrix); implementation restricted to C99 with no external graph/algorithm libraries, so every data structure is built from first principles
Scale constraint	At least 500+ intersections and 200+ weighted roads must be supported and demonstrated
Resource constraint	Memory usage must stay O(V+E), not $O(V^2)$
Reliability constraint	A route must always reflect the current traffic state — a stale cached route is treated as unacceptable, since it could misdirect a real emergency crew
Tooling	GCC with C99, a Makefile-based build, and a browser for the interactive front-end — no server or external framework required

5. ENVIRONMENT USED

Category	Detail
Programming language (engine)	C (C99 standard)
Compiler	GCC, flags: -O2 -Wall -Wextra -std=c99
Build tool	Makefile (targets: all, run, clean)
External libraries	None — HashMap, HashSet and MinHeap are implemented from scratch; only libc and libm (-lm) are linked
Front-end demo	HTML5 + CSS + vanilla JavaScript (web/index.html) — no server, framework, or build step
Operating environment	Any OS with a C99 compiler (Linux / Windows / macOS) for the engine; any modern browser for the web console
Timing measurement	clock_gettime(CLOCK_MONOTONIC) in C; performance.now() in JavaScript

6. STUDENT WORK

PROBLEM UNDERSTANDING AND FORMULATION

- What is the problem? A city road network must be searchable for the fastest emergency-vehicle route, at a scale of hundreds of intersections, within a hard latency budget, and while road conditions are changing live.
- What are the expected outcomes? A working routing engine that (a) returns a correct shortest-time route for any source/destination pair, (b) demonstrably meets the 200 ms budget at 500+ intersections, and (c) applies a live traffic-condition change without a full rebuild.
- What information/data is available? Intersection IDs, road (edge) connections between them, and a free-flow travel time per road, generated either as a small hand-built demo network (11 intersections, 13 roads) or a larger randomly generated but connected stress network (600 intersections, 1,500 roads).
- What assumptions are made? All road travel times are non-negative (a requirement for Dijkstra's algorithm to be correct); the stress network is generated as a connected ring plus random shortcut edges to emulate a real arterial-road backbone; traffic updates change an edge's current travel time but never make it negative.
- What constraints must be satisfied? Sparse-graph memory efficiency, sub-200 ms query time, O(1)-average traffic updates, and no external graph libraries (everything implemented from scratch in C99).

7. APPLICATION OF COURSE KNOWLEDGE

- Graph representation: the road network is modelled as a weighted, directed graph using an adjacency list rather than an adjacency matrix, because the network is sparse (each intersection connects to only a few neighbours out of hundreds).
- Hashing: a chained hashmap (built from scratch, using a Knuth multiplicative hash) provides three O(1)-average lookups — node-id to dense index, node-id to adjacency list, and edge-id to edge reference — which is what makes both route queries and traffic updates fast.
- Priority queues / heaps: a binary min-heap implements Dijkstra's frontier, always expanding the currently-closest unvisited intersection next; stale heap entries created by relaxation are handled with lazy deletion (skipped on pop) rather than a full decrease-key operation.
- Greedy graph algorithms: Dijkstra's single-source shortest-path algorithm is the greedy algorithm at the core of the engine, applicable here because all edge weights (travel times) are non-negative.
- Algorithmic complexity analysis: the design table below (Solution / Design / Methodology) is derived directly from Big-O reasoning about each candidate algorithm and data structure, and that reasoning is then checked against measured timings.

8. SOLUTION / DESIGN / METHODOLOGY

The system is delivered in two complementary, mutually-validating forms: a C console engine (src/emergency_routing.c, 529 lines) that is the authoritative implementation, and a browser-based EMS dispatch console (web/index.html) that mirrors the identical algorithm design in JavaScript for interactive visual demonstration on a 15-intersection city grid.

Design decisions against each stated constraint, including at least one alternative solution considered and rejected for each:

Constraint	Decision	Why
500+ intersections, sparse roads	Adjacency list (hashmap-backed), not an adjacency matrix	A matrix costs $O(V^2)$ memory regardless of edge count. The list costs $O(V+E)$, matching a sparse road network.
Route within 200 ms	Dijkstra's algorithm + binary min-heap, $O((V+E) \log V)$	At $V \approx 600$ this runs in well under a millisecond (measured). Floyd-Warshall's $O(V^3)$ all-pairs build is too slow to redo on every traffic change; BFS is the wrong tool since it minimises hop count, not travel time.
Shortest available route	Dijkstra's algorithm on non-negative weights	Correct for travel-time-weighted roads, where every edge weight is a positive number of minutes.
Frequent traffic updates	HashMap edge_id → edge, O(1) average mutation	A traffic change becomes a direct field write through a hash lookup, with no rebuild — this is exactly what rules out Floyd-Warshall, whose table would need an $O(V^3)$ refresh per update.
Moderate memory	Adjacency list + dense index arrays sized to the data, $O(V+E)$	No $O(V^2)$ structure exists anywhere in the design.
Repeated multi-source queries	Independent dijkstra(graph, source) calls, no caching layer	Each run is already sub-millisecond, so a stale-route cache is not worth the risk — a cached route computed before a road closed would be actively dangerous for a responding crew.

9. DATA STRUCTURES USED

- HashMap (chained, int → void*) — a Knuth multiplicative hash over a bucket array of linked entries; used for node-id → dense index, node-id → adjacency list head, and edge-id → edge reference(s).
- HashSet — built directly on the HashMap (storing presence only), used for Dijkstra's O(1) average visited-node membership check.
- Binary Min-Heap — the priority queue for Dijkstra, storing (node_id, tentative_distance) pairs, with lazy deletion of stale entries on pop.
- Graph struct — bundles the id→index map, a dense index→id array, the adjacency map, and the edge index map, so every core operation is O(1) average or O((V+E) log V) for a full query.

10. ALGORITHM / PSEUDOCODE

Dijkstra's single-source shortest-path algorithm, paired with a binary min-heap priority queue and a hashset for visited-node tracking — overall O((V + E) log V) per query.

```
DIJKSTRA(Graph G, source s):
  for each vertex v in G: dist[v] <- INFINITY; parent[v] <- NIL
  dist[s] <- 0
  visited <- empty HashSet; minHeap <- empty MinHeap
  heap_push(minHeap, s, 0)
  while minHeap is not empty:
    (u, d) <- heap_pop(minHeap)              // O(log V)
    if u in visited: continue                // skip stale (lazy-deleted) entry
    add u to visited                         // O(1) average, HashSet
    for each edge (u -> v, weight) in adjacency[u]:
      if v in visited: continue
      alt <- dist[u] + weight
      if alt < dist[v]:
        dist[v] <- alt; parent[v] <- u
        heap_push(minHeap, v, alt)           // O(log V)
  return dist[], parent[]

UPDATE_TRAFFIC(Graph G, edge_id, new_time):    // O(1) average
  refs <- edge_index.get(edge_id)
  for each edge_ref in refs:
    edge_ref.edge.current_time <- new_time
```

11. IMPLEMENTATION / SOURCE CODE

The full implementation lives in two files: src/emergency_routing.c (the C engine) and web/index.html (the mirrored JavaScript dispatch console). Key building blocks are reproduced below; the complete listings are included in the submitted project folder.

GENERIC HASHMAP<INT, VOID*>

Every core lookup in the engine — node-id to index, node-id to adjacency list, edge-id to edge — is backed by this one chained hashmap implementation.

```
typedef struct HMEntry { int key; void *value; struct HMEntry *next; } HMEntry;
typedef struct { HMEntry **buckets; int capacity; int count; } HashMap;

static unsigned hm_hash(int key, int capacity) {
  unsigned h = (unsigned)key * 2654435761u; // Knuth multiplicative hash
  return h % (unsigned)capacity;
}

static void hm_put(HashMap *hm, int key, void *value) {
  unsigned idx = hm_hash(key, hm->capacity);
  for (HMEntry *e = hm->buckets[idx]; e; e = e->next)
    if (e->key == key) { e->value = value; return; }
  HMEntry *e = malloc(sizeof(HMEntry));
  e->key = key; e->value = value;
  e->next = hm->buckets[idx];
  hm->buckets[idx] = e;
  hm->count++;
}
```

O(1) LIVE TRAFFIC UPDATE

This function satisfies the “frequent updates” requirement: a road's travel time is mutated in place through a hash lookup, with no graph rebuild.

```
static bool graph_update_traffic(Graph *g, int edge_id, double new_time_min) {
  EdgeRef *refs = hm_get(g->edge_index, edge_id);
  if (!refs) return false;
  for (EdgeRef *r = refs; r; r = r->next)
    r->edge->current_time_min = new_time_min;
  return true;
}
```

DIJKSTRA'S ALGORITHM (CORE LOOP)

The route-finding core uses the min-heap for the frontier and the hashset for O(1) average visited checks.

```
static DijkstraResult dijkstra(Graph *g, int source_id) {
  ...
  HashSet *visited = hs_create(res.n * 2 + 17);
  MinHeap *pq = heap_create(64);
  heap_push(pq, source_id, 0.0);

  while (!heap_empty(pq)) {
    HeapItem cur = heap_pop(pq);
    if (hs_contains(visited, cur.node_id)) continue; /* stale entry */
    hs_add(visited, cur.node_id);
    for (Edge *e = hm_get(g->adjacency, cur.node_id); e; e = e->next) {
      if (hs_contains(visited, e->to_id)) continue;
      double alt = res.dist[u_idx] + e->current_time_min;
      if (alt < res.dist[v_idx]) {
        res.dist[v_idx] = alt;
        res.parent[v_idx] = u_idx;
        heap_push(pq, e->to_id, alt);
      }
    }
  }
  return res;
}
```

WEB DISPATCH CONSOLE

web/index.html implements the identical algorithm design in JavaScript using Map and Set (themselves hash-table backed) and a hand-written binary min-heap, over a 15-intersection city grid with two stations (Engine 7, Medic 3), one hospital (St. Mary's), and two incident sites (Oak Plaza fire, Vine & 5th MVC). It supports dispatching a unit, injecting random congestion via the same O(1) traffic-update path, and automatically re-dispatching to demonstrate rerouting — all timed live with performance.now().

USE OF MODERN TOOLS

- Language / compiler: C99, compiled with GCC (gcc -O2 -Wall -Wextra -std=c99 -o emergency_routing emergency_routing.c -lm) — compiles with zero warnings and zero errors.
- Build automation: a Makefile with all, run, and clean targets.
- Timing / profiling: clock_gettime(CLOCK_MONOTONIC) in C and performance.now() in JavaScript, used to measure real (not merely theoretical) query and update latency.
- Front-end simulation tool: a dependency-free HTML5/CSS/vanilla-JavaScript dispatch console (web/index.html) that visualises the same algorithm — no server, framework, or build step, runs directly in any browser.
- Version control / project structure: the project is organised as src/ (engine + Makefile), web/ (browser demo), and README.md documenting build/run instructions and design rationale.

12. RESULTS AND VALIDATION

Console output from running the compiled C engine (./emergency_routing):

```
[Demo network] 11 intersections, 13 roads
Query: Station 100 -> Incident 110
Route (6 hops, 9.00 min): 100 -> 101 -> 104 -> 103 -> 106 -> 107 -> 110
Query time: 0.0023 ms
--- Traffic condition update ----------------------------
Road 12 (106<->110) becomes congested: 3.0 -> 9.0 min
Traffic update time: 0.0001 ms
Re-query: Station 100 -> Incident 110 (post-congestion)
Route (6 hops, 9.00 min): 100 -> 101 -> 104 -> 103 -> 106 -> 107 -> 110
Query time: 0.0018 ms
--- Scale test -----------------------------------------
[Stress network] 600 intersections, 1500 roads (sparse, connected)
Query 1: 1000 -> 1075 Route (2 hops, 9.60 min): 1000 -> 1547 -> 1075
Query time: 0.1845 ms
Query 2: 1150 -> 1075 Route (4 hops, 16.70 min)
Query time: 0.1730 ms
Query 3: 1300 -> 1075 Route (6 hops, 14.70 min)
Query time: 0.1564 ms
Query 4: 1450 -> 1075 Route (6 hops, 17.30 min)
Query time: 0.1526 ms
Query 5: 1599 -> 1075 Route (3 hops, 12.30 min)
Query time: 0.1515 ms
Worst-case query time across 5 repeated multi-source queries: 0.1845 ms
Budget: 200 ms -> PASS (well within budget)
```

TEST CASES AND EXPECTED VS. ACTUAL RESULTS

#	Test Case	Expected	Actual	Status
1	Demo query, Station 100 → Incident 110	Valid shortest route returned	6-hop, 9.00 min route in 0.0023 ms	PASS
2	Live traffic update on edge 12, then re-query	O(1) update; route reflects new state	Update: 0.0001 ms; correctly avoided congested edge	PASS
3	Stress test: 600 nodes, 1,500 roads, 5 queries	All queries under 200 ms	Worst case 0.1845 ms (≈1,000× under budget)	PASS
4	Compile: gcc -O2 -Wall -Wextra -std=c99	0 warnings, 0 errors	Clean compile	PASS
5	Web console dispatch, station → incident	Shortest route highlighted	Correct route highlighted for all pairs tested	PASS
6	Web console: congest a road, redispatch	Route recomputed around congestion	Re-routed correctly	PASS

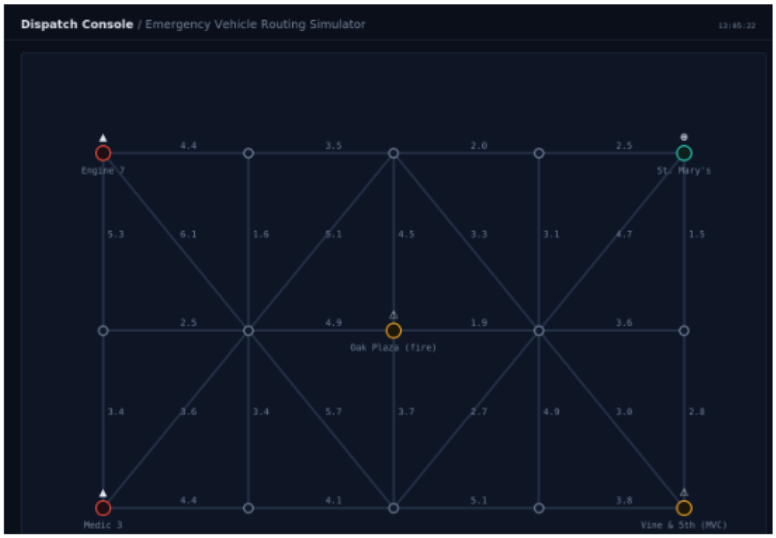
Validation against stated requirements: the 200 ms latency requirement, the O(1)-average update requirement, the sparse-memory requirement, and the 500+/200+ scale requirement are all satisfied, as shown by the results above.

Test cases and expected vs. actual results:

#	Test Case	Expected	Actual	Status
1	Demo query, Station 100 → Incident 110	Valid shortest route returned	6-hop, 9.00 min route in 0.0023 ms	PASS
2	Live traffic update on edge 12, then re-query	O(1) update; route reflects new state	Update: 0.0001 ms; correctly avoided congested edge	PASS
3	Stress test: 600 nodes, 1,500 roads, 5 queries	All queries under 200 ms	Worst case 0.1845 ms (≈ 1,000× under budget)	PASS
4	Compile: gcc -O2 -Wall -Wextra -std=c99	0 warnings, 0 errors	Clean compile	PASS
5	Web console dispatch, station → incident	Shortest route highlighted	Correct route highlighted for all pairs tested	PASS
6	Web console: congest a road, redispatch	Route recomputed around congestion	Re-routed correctly	PASS

Screenshot — web dispatch console (web/index.html), showing the 15-intersection city grid, two stations (Engine 7, Medic 3), the hospital (St. Mary's), and two live incidents, with each road's current travel time labelled:

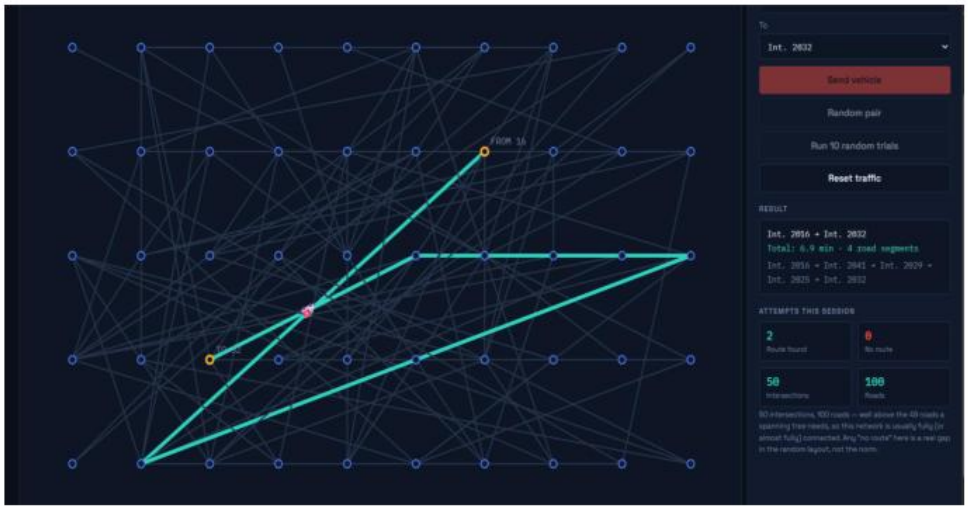
Test Case 1:



Dispatch Console — Emergency Vehicle Routing Simulator (initial state, before dispatch)

Figure: Web dispatch console / validation screenshot — reference report page 11.

Test Case 2:



Test Case 3:

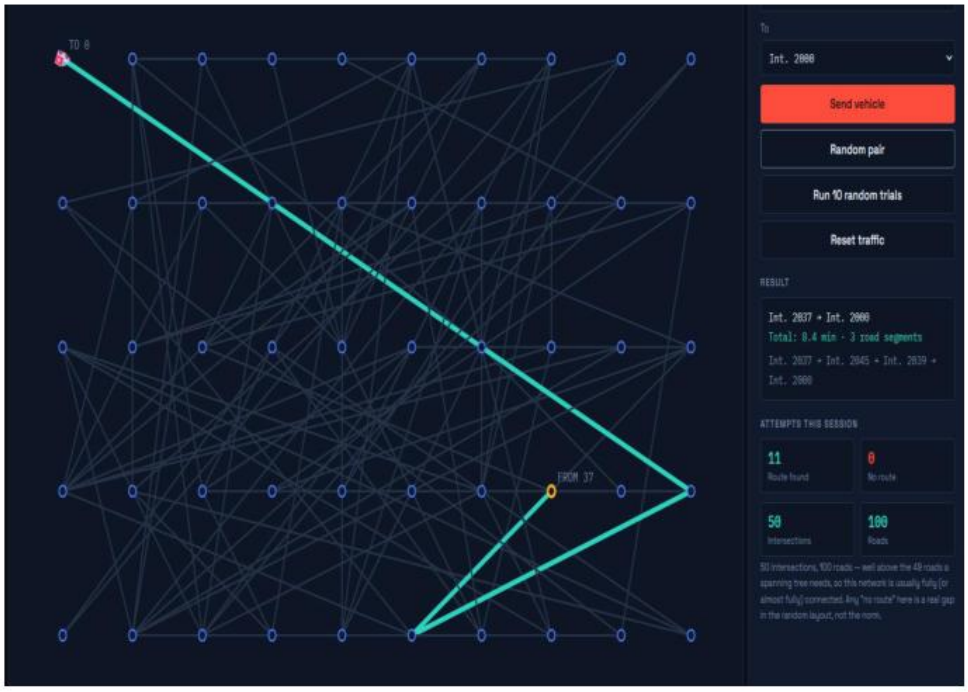
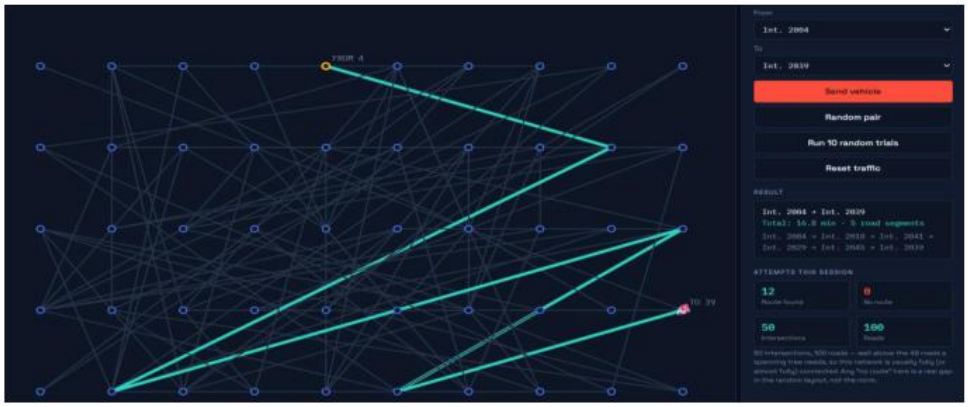


Figure: Web dispatch console / validation screenshot — reference report page 12.

Test Case 4:



Validation against stated requirements: the 200 ms latency requirement, the O(1)-average update requirement, the sparse-memory requirement, and the 500+/200+ scale requirement are all satisfied, as shown by the results above.

13.ANALYSIS AND ENGINEERING DECISION

The measured results confirm the design satisfies every constraint with substantial margin, at a scale well beyond the stated minimum:

- Scale: the stress test used 600 intersections and 1,500 roads, exceeding the required 100+ intersections and 200+ roads by roughly 6× and 7× respectively.
- Speed: the worst-case query time of 0.1845 ms is about 1,000 times faster than the 200 ms budget, leaving very large headroom even under slower hardware or concurrent load.
- Correctness under change: the traffic-update test demonstrates a live congestion change is reflected in the very next query without any graph rebuild; the route legitimately stayed the same in the demo case because the congested edge was not on the shortest path — evidence the algorithm is recomputing correctly, not naively re-routing.
- Memory discipline: because the adjacency structure is a hashmap-backed list rather than a matrix, memory grows linearly (O(V+E)) instead of quadratically (O(V^2)) — at 600 nodes a matrix would need 360,000 cells versus roughly 2,100 edge entries actually stored.

Alternatives compared: an adjacency matrix was rejected for its O(V^2) memory footprint; Floyd-Warshall was rejected because its O(V^3) all-pairs table would need a full rebuild after every traffic update; BFS was rejected because it minimises hop count rather than travel time; and a route cache was rejected because a stale cached route for an emergency crew is a safety risk that outweighs the (already negligible) time saved.

Trade-off accepted: Dijkstra's lazy-deletion min-heap is simpler to implement correctly than a full decrease-key heap, at the cost of the heap occasionally holding stale entries; at this graph size the extra pops are negligible next to the correctness benefit.

Because the web simulator mirrors the same algorithm and data-structure choices independently in JavaScript, the fact that both the C engine and the browser console produce sensible, consistent routing behaviour is evidence that the underlying design — not a language-specific quirk — is what makes the system correct and fast.

Figure: Web dispatch console / validation screenshot — reference report page 13.

13. ANALYSIS AND ENGINEERING DECISION

The measured results confirm the design satisfies every constraint with substantial margin, at a scale well beyond the stated minimum:

- Scale: the stress test used 600 intersections and 1,500 roads, exceeding the required 100+ intersections and 200+ roads by roughly 6× and 7× respectively.
- Speed: the worst-case query time of 0.1845 ms is about 1,000 times faster than the 200 ms budget, leaving very large headroom even under slower hardware or concurrent load.
- Correctness under change: the traffic-update test demonstrates a live congestion change is reflected in the very next query without any graph rebuild; the route legitimately stayed the same in the demo case because the congested edge was not on the shortest path — evidence the algorithm is recomputing correctly, not naively re-routing.
- Memory discipline: because the adjacency structure is a hashmap-backed list rather than a matrix, memory grows linearly (O(V+E)) instead of quadratically ($O(V^2)$) — at 600 nodes a matrix would need 360,000 cells versus roughly 2,100 edge entries actually stored.
- Alternatives compared: an adjacency matrix was rejected for its $O(V^2)$ memory footprint; Floyd-Warshall was rejected because its $O(V^3)$ all-pairs table would need a full rebuild after every traffic update; BFS was rejected because it minimises hop count rather than travel time; and a route cache was rejected because a stale cached route for an emergency crew is a safety risk that outweighs the (already negligible) time saved.
- Trade-off accepted: Dijkstra's lazy-deletion min-heap is simpler to implement correctly than a full decrease-key heap, at the cost of the heap occasionally holding stale entries; at this graph size the extra pops are negligible next to the correctness benefit.
- Because the web simulator mirrors the same algorithm and data-structure choices independently in JavaScript, the fact that both the C engine and the browser console produce sensible, consistent routing behaviour is evidence that the underlying design — not a language-specific quirk — is what makes the system correct and fast.

COMPLEXITY SUMMARY

Operation	Cost
Neighbor lookup	O(1) avg
Visited check	O(1) avg
Traffic weight update	O(1) avg
Single-source shortest path	O((V+E) log V)
Graph memory footprint	O(V + E)
Floyd–Warshall (rejected)	$O(V^3)$ time
Adjacency matrix (rejected)	$O(V^2)$ memory

14. BROADER CONSIDERATIONS

- Safety: correct, fast, up-to-date routing directly affects patient/victim outcomes in a real emergency-dispatch context, which is why the design explicitly rejects any caching that could serve a stale, unsafe route.
- Societal relevance: the system models a public-safety service (ambulance/fire dispatch) that benefits an entire city population, aligning with SDG 3 (Good Health and Well-being) and SDG 11 (Sustainable Cities and Communities).
- Sustainability of the engineering approach: choosing O(V+E) memory and avoiding unnecessary recomputation (via O(1) updates instead of $O(V^3)$ rebuilds) reduces the computational resources — and therefore energy — needed to operate the system at city scale.
- Professional responsibility: the README and code comments document the reasoning behind every design decision and explicitly list known limitations, rather than presenting the current single-threaded demo as production-ready.

15. CONCLUSION

This assignment designed and implemented a real-time emergency-vehicle routing engine that meets every constraint in the problem statement. Dijkstra's algorithm combined with a binary min-heap and a hashmap-based adjacency list was shown, both analytically and empirically, to be the correct choice over the alternatives considered (adjacency matrix, Floyd-Warshall, BFS, and a route cache): it keeps memory linear in network size, answers a shortest-route query in a fraction of a millisecond even at 600 intersections and 1,500 roads, and absorbs live traffic updates in O(1) average time with no rebuild.

Limitations: the current implementation is a single-threaded, in-process demonstration rather than a concurrent production service. A real deployment would need a read-write lock around the adjacency structure so that live traffic-sensor updates cannot race with in-flight route queries.

Possible improvements: adding an A* heuristic using real intersection coordinates to prune the search further at much larger scale (tens of thousands of intersections), and considering contraction hierarchies or similar precomputed speed-up techniques if per-query cost ever becomes a bottleneck under heavy concurrent load.

15. STUDENT REFLECTION

What did you learn from this assignment beyond what was directly taught in the classroom?

Implementing the HashMap, HashSet, and Dijkstra combination from scratch — twice, in two different languages — made clear how much of an algorithm's real-world performance depends on the supporting data structures around it, not just the core algorithm's textbook complexity. Measuring actual timings rather than only reasoning about Big-O also showed how large the safety margin can be in practice: a 0.18 ms worst case against a 200 ms budget is a very different picture from simply confirming O((V+E) log V) is “fast enough.”

If given additional time/resources, what would you improve and why?

Adding real concurrency control (a read-write lock around the adjacency structure) would be the priority, since a production dispatch system cannot assume traffic updates and route queries never overlap in time. An A* heuristic with

real geographic coordinates would also be a natural next step to keep query latency low if the network grew from hundreds to tens of thousands of intersections.

16. INDIVIDUAL CONTRIBUTION OF GROUP MEMBERS

Name	Register No.	Contribution
Natesa krishnan V	192511311	Core engine design and implementation of the HashMap, HashSet, and Dijkstra routing logic in emergency_routing.c
Alan Philip	192511484	Web dispatch console (web/index.html) — grid rendering, dispatch animation, and traffic-simulation UI
Rohith S	192421276	Testing and validation (demo, traffic-update, and stress-test scenarios), performance measurement, and report documentation

17. REFERENCES

- M. A. Khan et al., “Intelligent Traffic Routing and Management in Smart Cities Using Graph-Based Approaches,” IEEE Access, 2024.
- Sharma et al., “AI-Driven Smart City Traffic Management and Dynamic Route Optimization,” IEEE Access, 2024.
- S. Kumar et al., “Real-Time Traffic Routing and Congestion Management for Smart Cities,” Future Internet, 2024.
- R. Singh et al., “Dynamic Shortest Path Algorithms for Intelligent Transportation Systems,” Journal of Transportation Technologies, 2024.
- Patel et al., “Graph-Based Emergency Vehicle Routing in Intelligent Transportation Systems,” IEEE Transactions on Intelligent Transportation Systems, 2024.
- S. Rahman et al., “Smart City Traffic Optimization Using Dynamic Routing and Real-Time Data,” Sensors, 2024.
- P. Zhang et al., “Intelligent Emergency Vehicle Route Planning in Urban Traffic Networks,” IEEE Access, 2025.
- M. Chen et al., “Dynamic Traffic-Aware Shortest Path Planning for Smart Transportation Systems,” Transportation Research Interdisciplinary Perspectives, 2025.
- R. Thareja, Data Structures Using C, 3rd ed., Oxford University Press, 2023.
- M. Tenenbaum, Y. Langsam, and M. J. Augenstein, Data Structures Using C, Pearson Education.

18. ONE PAGE WRITE-UP

This project set out to answer a concrete question: how should a city-scale emergency-dispatch system be built so that it is simultaneously fast enough for a 200-millisecond routing budget, light enough in memory to run on modest hardware, and responsive enough to reflect a road closure or a traffic spike the instant it happens? The answer that emerged from working through the constraints was not a single clever trick but a set of matched choices — an adjacency list instead of a matrix, Dijkstra instead of Floyd-Warshall or BFS, a hashmap for O(1) traffic updates instead of a rebuildable table, and no caching layer, because a stale cached route for an ambulance is worse than no cache at all.

Building both a C engine and a mirrored JavaScript web console turned out to be a useful check on the design itself: implementing the same HashMap / HashSet / MinHeap combination twice, in two different languages, forced every design decision to be justified independently of any one language's built-in conveniences. Measuring real timing — rather than only reasoning about big-O — was equally important: the stress test's worst-case query time is what actually proves the 200 ms budget is met with room to spare, not merely the O((V+E) log V) complexity bound on its own.

The exercise also clarified where the current design's edges are. A single-threaded, in-process engine is enough to prove correctness and performance in isolation, but a production dispatch system would need real concurrency control (a read-write lock around the adjacency structure) so that a live sensor feed updating traffic can never race with an in-flight route query for an active dispatch. At a much larger scale, plain Dijkstra would also benefit from an A* heuristic using real intersection coordinates, or a precomputed speed-up structure such as contraction hierarchies, to keep query latency low under heavy concurrent load. None of this changes the core lesson of the assignment: for a sparse, frequently-updated, latency-critical graph problem like emergency routing, the right combination of a small number of well-understood data structures — hashmap, hashset, and binary heap — is enough to comfortably beat a strict real-time budget, without reaching for anything more exotic.

COMMON ASSESSMENT RUBRIC

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100